In [1]:
```
import pandas as pd
import numpy as  np
import matplotlib.pyplot as plt
```

In [2]:
```
df=pd.read_csv("Stud_per.csv")
```

In [3]:
```
df.head()
```

Out[3]:

	Math_score	Reading_score	Writing_score	Placement_score	Club_join_date	Placeme
0	77.0	91.0	70.0	81.0	2024.0	
1	73.0	90.0	61.0	90.0	2021.0	
2	70.0	86.0	77.0	89.0	2022.0	
3	66.0	NaN	NaN	83.0	2022.0	
4	69.0	79.0	73.0	85.0	2023.0	

In [4]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 49 entries, 0 to 48
Data columns (total 6 columns):
 #   Column                 Non-Null Count  Dtype
---  ------                 --------------  -----
 0   Math_score             43 non-null     float64
 1   Reading_score          38 non-null     float64
 2   Writing_score          40 non-null     float64
 3   Placement_score        39 non-null     float64
 4   Club_join_date         39 non-null     float64
 5   Placement_offer_count  46 non-null     float64
dtypes: float64(6)
memory usage: 2.4 KB
```

In [5]:
```
df.isnull().sum().sum()
```

Out[5]:
```
np.int64(49)
```

In [6]:
```
df.describe()
```

Out[6]:

	Math_score	Reading_score	Writing_score	Placement_score	Club_join_date	Plac
count	43.000000	38.000000	40.000000	39.000000	39.000000	
mean	71.441860	85.421053	69.950000	85.487179	2021.846154	
std	7.034955	6.549733	5.991233	9.136024	2.084258	
min	62.000000	75.000000	60.000000	61.000000	2015.000000	
25%	66.000000	80.250000	65.000000	80.000000	2021.000000	
50%	70.000000	85.500000	70.000000	86.000000	2022.000000	
75%	77.500000	90.750000	74.250000	91.000000	2023.000000	
max	90.000000	100.000000	80.000000	100.000000	2024.000000	

In [7]:
```
df.notnull().sum()
```

Out[7]:
```
Math_score              43
Reading_score           38
Writing_score           40
Placement_score         39
Club_join_date          39
Placement_offer_count   46
dtype: int64
```

In [8]:
```
df.columns
```

Out[8]:
```
Index(['Math_score', 'Reading_score', 'Writing_score', 'Placement_score',
       'Club_join_date', 'Placement_offer_count'],
      dtype='object')
```

In [11]:
```
df.drop([40,41,42,45,46],inplace=True)
```

In [12]:
```
df
```

Out[12]:

	Math_score	Reading_score	Writing_score	Placement_score	Club_join_date	Placem
0	77.0	91.0	70.0	81.0	2024.0	
1	73.0	90.0	61.0	90.0	2021.0	
2	70.0	86.0	77.0	89.0	2022.0	
3	66.0	NaN	NaN	83.0	2022.0	
4	69.0	79.0	73.0	85.0	2023.0	
5	62.0	86.0	72.0	77.0	2023.0	
6	78.0	94.0	60.0	NaN	NaN	
7	NaN	88.0	79.0	NaN	NaN	
8	NaN	80.0	75.0	NaN	NaN	
9	78.0	88.0	61.0	NaN	NaN	
10	67.0	78.0	66.0	NaN	NaN	
11	74.0	77.0	80.0	99.0	2022.0	
12	74.0	NaN	63.0	92.0	2023.0	
13	74.0	NaN	74.0	85.0	2021.0	
14	78.0	NaN	77.0	95.0	2024.0	
15	66.0	76.0	72.0	81.0	2020.0	
16	74.0	78.0	65.0	95.0	2020.0	
17	66.0	94.0	70.0	88.0	2020.0	
18	78.0	95.0	62.0	100.0	NaN	
19	75.0	87.0	63.0	89.0	2023.0	
20	79.0	84.0	78.0	99.0	2021.0	
21	71.0	95.0	72.0	75.0	2024.0	
22	90.0	93.0	68.0	81.0	2022.0	
23	65.0	83.0	NaN	100.0	2024.0	
24	62.0	85.0	NaN	97.0	2023.0	
25	68.0	87.0	NaN	99.0	2021.0	
26	65.0	86.0	79.0	88.0	2021.0	
27	75.0	86.0	65.0	77.0	2024.0	
28	69.0	100.0	66.0	84.0	2023.0	
29	62.0	76.0	62.0	61.0	2022.0	
30	67.0	92.0	69.0	86.0	2020.0	
31	NaN	NaN	69.0	79.0	2021.0	
32	NaN	NaN	78.0	86.0	2024.0	

	Math_score	Reading_score	Writing_score	Placement_score	Club_join_date	Placem
33	80.0	81.0	65.0	78.0	NaN	
34	80.0	81.0	70.0	75.0	NaN	
35	78.0	75.0	72.0	65.0	2021.0	
36	75.0	95.0	73.0	75.0	2024.0	
37	65.0	92.0	80.0	92.0	2022.0	
38	89.0	82.0	65.0	89.0	2015.0	
39	66.0	84.0	65.0	78.0	2024.0	
43	68.0	85.0	64.0	82.0	2015.0	
44	78.0	76.0	77.0	85.0	2021.0	
47	62.0	81.0	71.0	87.0	2023.0	
48	62.0	80.0	70.0	87.0	2022.0	

In [13]:
```
df.isna().sum()
```

Out[13]:
```
Math_score               4
Reading_score            6
Writing_score            4
Placement_score          5
Club_join_date           8
Placement_offer_count    3
dtype: int64
```

In [14]:
```
df["Math_score"].fillna(value=df["Math_score"].mean(),inplace=True)
```

```
<ipython-input-14-a7dc3b4e39a6>:1: FutureWarning: A value is trying to be set on
a copy of a DataFrame or Series through chained assignment using an inplace metho
d.
The behavior will change in pandas 3.0. This inplace method will never work becau
se the intermediate object on which we are setting values always behaves as a cop
y.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.meth
od({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to pe
rform the operation inplace on the original object.


  df["Math_score"].fillna(value=df["Math_score"].mean(),inplace=True)
```

In [15]:
```
df
```

Out[15]:

	Math_score	Reading_score	Writing_score	Placement_score	Club_join_date	Placem
0	77.000	91.0	70.0	81.0	2024.0	
1	73.000	90.0	61.0	90.0	2021.0	
2	70.000	86.0	77.0	89.0	2022.0	
3	66.000	NaN	NaN	83.0	2022.0	
4	69.000	79.0	73.0	85.0	2023.0	
5	62.000	86.0	72.0	77.0	2023.0	
6	78.000	94.0	60.0	NaN	NaN	
7	71.875	88.0	79.0	NaN	NaN	
8	71.875	80.0	75.0	NaN	NaN	
9	78.000	88.0	61.0	NaN	NaN	
10	67.000	78.0	66.0	NaN	NaN	
11	74.000	77.0	80.0	99.0	2022.0	
12	74.000	NaN	63.0	92.0	2023.0	
13	74.000	NaN	74.0	85.0	2021.0	
14	78.000	NaN	77.0	95.0	2024.0	
15	66.000	76.0	72.0	81.0	2020.0	
16	74.000	78.0	65.0	95.0	2020.0	
17	66.000	94.0	70.0	88.0	2020.0	
18	78.000	95.0	62.0	100.0	NaN	
19	75.000	87.0	63.0	89.0	2023.0	
20	79.000	84.0	78.0	99.0	2021.0	
21	71.000	95.0	72.0	75.0	2024.0	
22	90.000	93.0	68.0	81.0	2022.0	
23	65.000	83.0	NaN	100.0	2024.0	
24	62.000	85.0	NaN	97.0	2023.0	
25	68.000	87.0	NaN	99.0	2021.0	
26	65.000	86.0	79.0	88.0	2021.0	
27	75.000	86.0	65.0	77.0	2024.0	
28	69.000	100.0	66.0	84.0	2023.0	
29	62.000	76.0	62.0	61.0	2022.0	
30	67.000	92.0	69.0	86.0	2020.0	
31	71.875	NaN	69.0	79.0	2021.0	
32	71.875	NaN	78.0	86.0	2024.0	

	Math_score	Reading_score	Writing_score	Placement_score	Club_join_date	Placem
33	80.000	81.0	65.0	78.0	NaN	
34	80.000	81.0	70.0	75.0	NaN	
35	78.000	75.0	72.0	65.0	2021.0	
36	75.000	95.0	73.0	75.0	2024.0	
37	65.000	92.0	80.0	92.0	2022.0	
38	89.000	82.0	65.0	89.0	2015.0	
39	66.000	84.0	65.0	78.0	2024.0	
43	68.000	85.0	64.0	82.0	2015.0	
44	78.000	76.0	77.0	85.0	2021.0	
47	62.000	81.0	71.0	87.0	2023.0	
48	62.000	80.0	70.0	87.0	2022.0	

In [16]:
```
df.isnull().sum()
```

Out[16]:
```
Math_score               0
Reading_score            6
Writing_score            4
Placement_score          5
Club_join_date           8
Placement_offer_count    3
dtype: int64
```

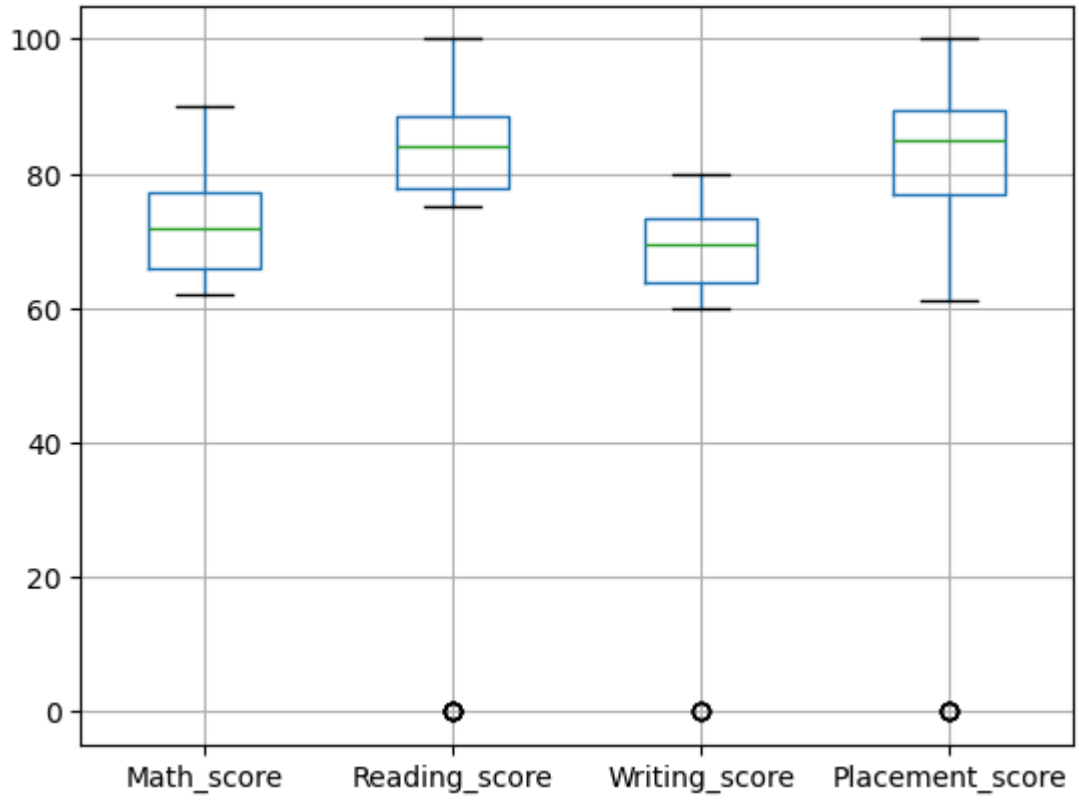
In [20]:
```
df.fillna(0,inplace=True)
```

In [21]:
```
df.isnull().sum()
```

Out[21]:
```
Math_score               0
Reading_score            0
Writing_score            0
Placement_score          0
Club_join_date           0
Placement_offer_count    0
dtype: int64
```

In [22]:
```
col=['Math_score', 'Reading_score', 'Writing_score', 'Placement_score']
df.boxplot(col)
```

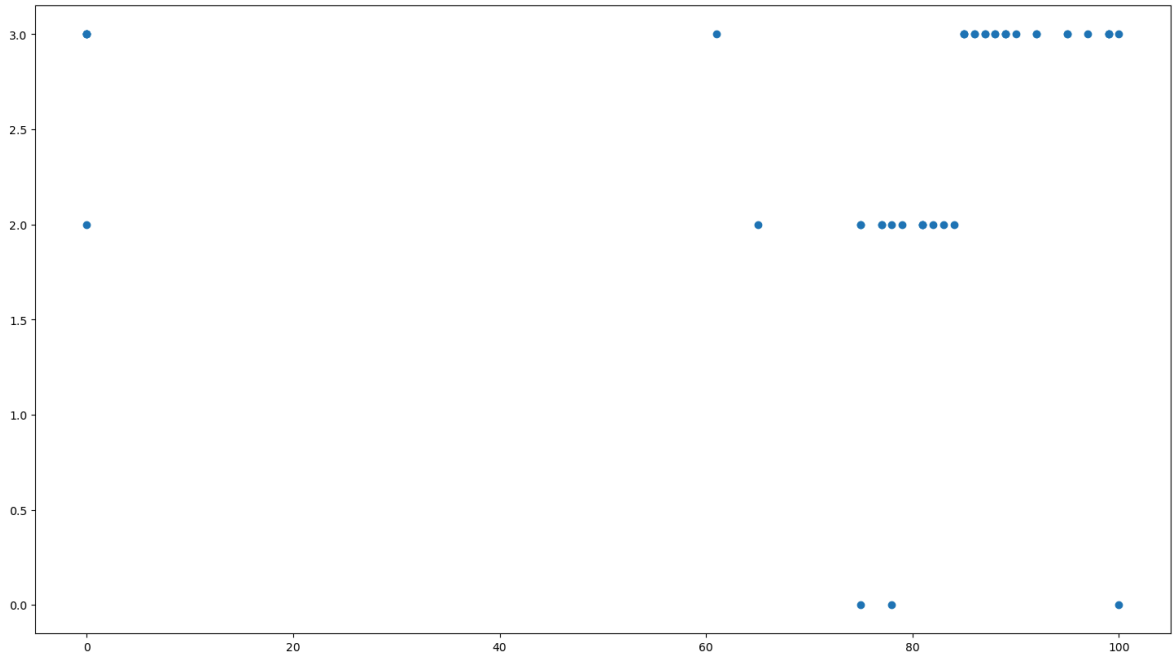
Out[22]:
```
<Axes: >
```



In [23]:
```
print(np.where(df["Math_score"]>90))
```

```
(array([], dtype=int32),)
```

In [24]:
```
fig, ax = plt.subplots(figsize = (18,10))
ax.scatter(df['Placement_score'], df['Placement_offer_count'])
plt.show()
```



In [25]:
```
print(np.where((df['Placement_score']<50) & (df['Placement_offer_count']>1)))
```

```
(array([ 6,  7,  8,  9, 10]),)
```

In [26]:
```
from scipy import stats
```

In [27]:
```
z=np.abs(stats.zscore(df['Math_score']))
```

In [28]:
```
print(z)
```

```
0     0.765854
1     0.168114
2     0.280190
3     0.877930
4     0.429625
5     1.475669
6     0.915288
7     0.000000
8     0.000000
9     0.915288
10    0.728495
11    0.317549
12    0.317549
13    0.317549
14    0.915288
15    0.877930
16    0.317549
17    0.877930
18    0.915288
19    0.466984
20    1.064723
21    0.130755
22    2.708507
23    1.027365
24    1.475669
25    0.579060
26    1.027365
27    0.466984
28    0.429625
29    1.475669
30    0.728495
31    0.000000
32    0.000000
33    1.214158
34    1.214158
35    0.915288
36    0.466984
37    1.027365
38    2.559072
39    0.877930
43    0.579060
44    0.915288
47    1.475669
48    1.475669
Name: Math_score, dtype: float64
```

In [29]:
```
threshold = 0.18
```

In [30]:
```
outliers=np.where(z<threshold)
outliers
```

Out[30]:
```
(array([ 1,  7,  8, 21, 31, 32]),)
```

In [31]:
```
sorted_rscore= sorted(df['Reading_score'])
```

In [32]:
```
sorted_rscore
```

Out[32]:
```
[0.0,
 0.0,
 0.0,
 0.0,
 0.0,
 0.0,
 75.0,
 76.0,
 76.0,
 76.0,
 77.0,
 78.0,
 78.0,
 79.0,
 80.0,
 80.0,
 81.0,
 81.0,
 81.0,
 82.0,
 83.0,
 84.0,
 84.0,
 85.0,
 85.0,
 86.0,
 86.0,
 86.0,
 86.0,
 87.0,
 87.0,
 88.0,
 88.0,
 90.0,
 91.0,
 92.0,
 92.0,
 93.0,
 94.0,
 94.0,
 95.0,
 95.0,
 95.0,
 100.0]
```

In [33]:
```
q1=np.percentile(sorted_rscore,25)
q3=np.percentile(sorted_rscore,75)
```

In [34]:
```
print(q1,q3)
```

```
77.75 88.5
```

In [36]:
```
IQR = q3-q1
```

In [37]:
```
lwr_bound=q1-(1.5*IQR)
upr_bound=q3+(1.5*IQR)
print(lwr_bound, upr_bound)
```

```
61.625 104.625
```

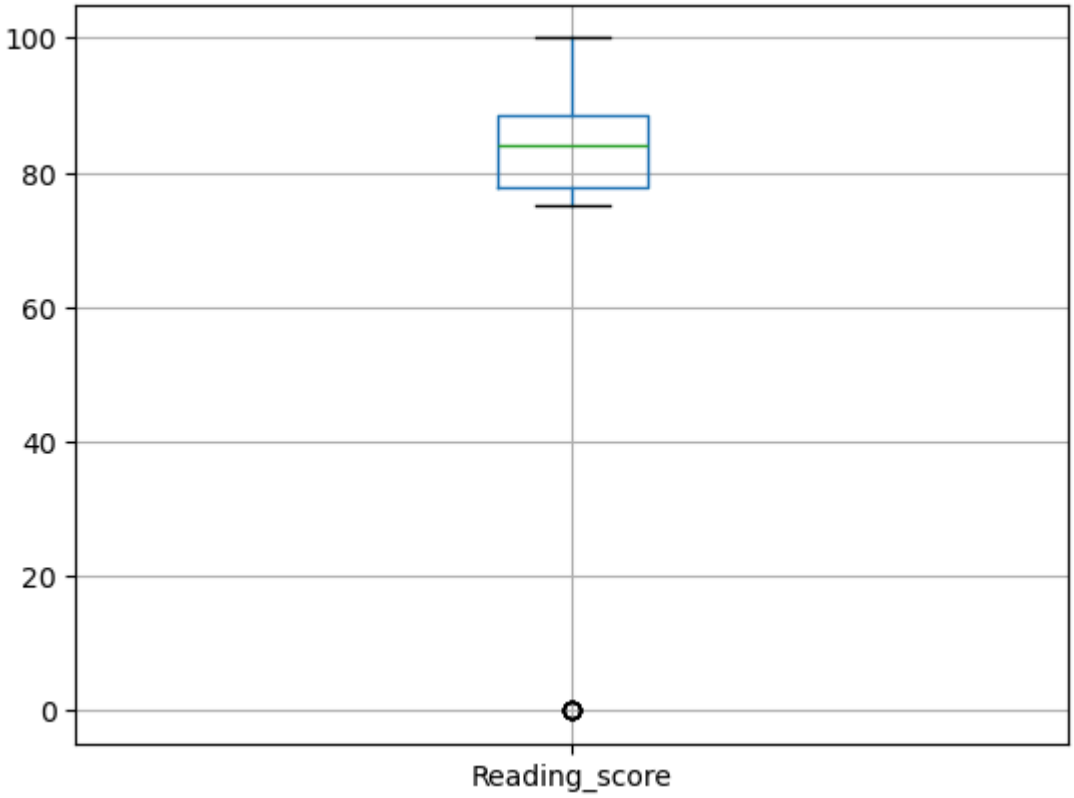
In [38]:
```
r_outliers = []
for i in sorted_rscore:
    if (i<lwr_bound or i>upr_bound):
        r_outliers.append(i)
print(r_outliers)
```

```
[0.0, 0.0, 0.0, 0.0, 0.0, 0.0]
```

In [39]:
```
col1=['Reading_score']
df.boxplot(col1)
```

Out[39]:
```
<Axes: >
```

In [40]:
```
plt.show()
```

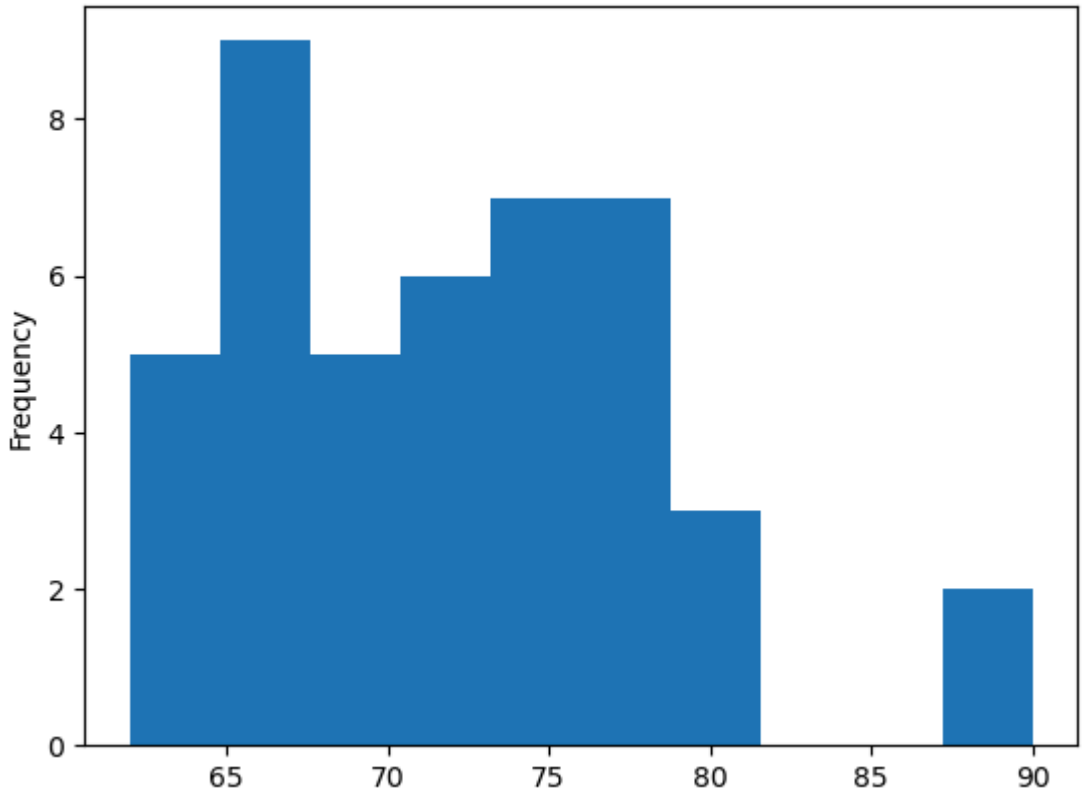


In [41]:
```
import matplotlib.pyplot as plt
```

In [45]:
```
df['Math_score'].plot(kind='hist')
```

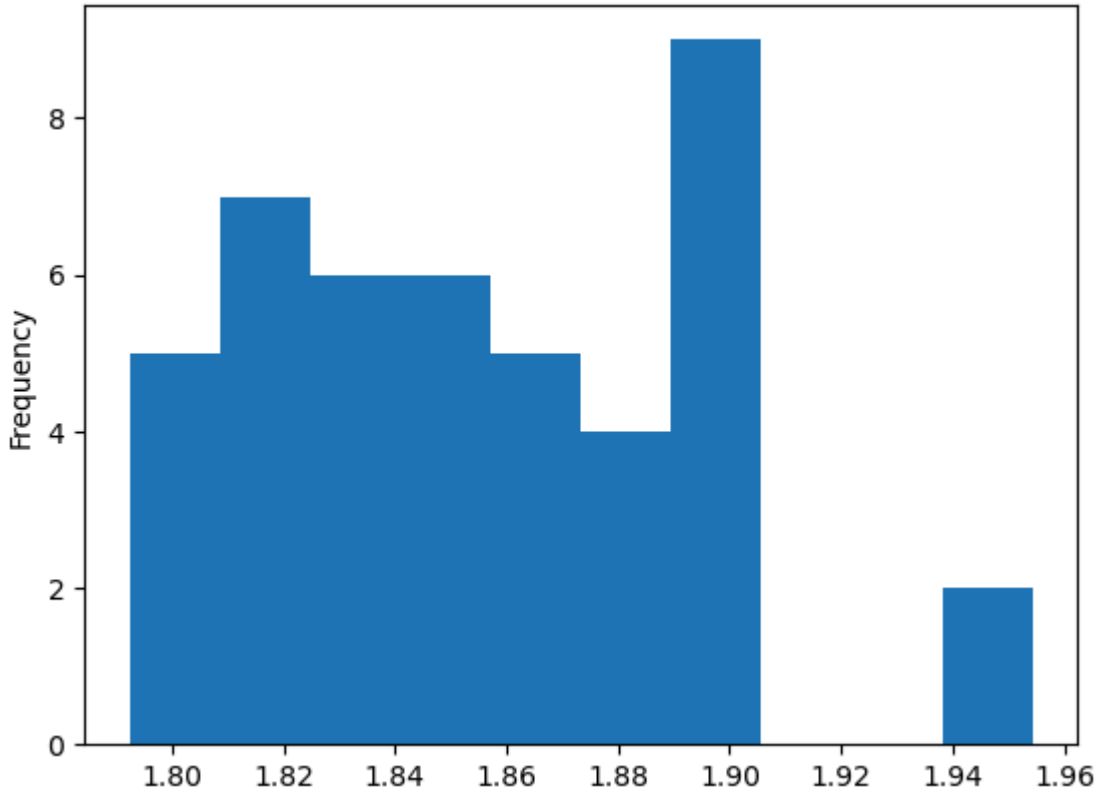
Out[45]:
```
<Axes: ylabel='Frequency'>
```

In [46]:
```
plt.show()
```



In [49]:
```
df['log_math'] = np.log10(df['Math_score'])
```

In [50]:
```
df['log_math'].plot(kind ='hist')
```

Out[50]:
```
<Axes: ylabel='Frequency'>
```

In [51]:
```
plt.show()
```